

智能在线编程练习平台

项目总结报告

课程名称：软件项目管理

项目名称：智能在线编程练习平台（智能算法题库与在线评测系统）

团队成员：李俊熠、冯朝航、胡阳、黄嘉鑫、孙赫、唐代鑫、谢浩敏

代码仓库：SPM-PSP / SPM26-2

报告日期：2026 年 5 月 31 日

2 项目总结报告

2.1 绪论

2.1.1 项目背景

在计算机专业教学、求职笔试面试、算法竞赛训练等核心场景中，算法编程能力是衡量开发者技术水平的关键指标。随着编程学习需求的持续增长，传统算法题库系统逐渐暴露出诸多痛点：功能单一，仅支持基础的题目展示与结果判定；评测机制僵化，仅验证代码正确性，无法对代码质量、算法效率进行深度分析；题目资源固化，无法满足不同基础用户的个性化练习诉求；缺乏专业的指导能力，学习者难以自主优化代码、提升编程规范。

为破解上述行业痛点，本项目聚焦算法学习全流程，研发了“智能在线编程练习平台”（智能算法题库与在线评测系统）。平台以算法学习为核心，融合自研 C++ 在线判题（OJ）引擎与人工智能大模型技术，构建了集题目练习、代码提交、自动测评、AI 智能分析、题目自动生成于一体的全流程智能化平台，采用“传统 OJ 判题 + AI 智能赋能”的双引擎模式，为在校学生、编程爱好者、求职备考人员、算法竞赛选手等全层次用户提供一站式、高效率的算法学习解决方案，同时助力高校实现智能化教学辅助。

2.1.2 相关技术现状

在线判题（Online Judge, OJ）系统经过多年发展已较为成熟，国内外存在 LeetCode、洛谷、Codeforces、HustOJ 等代表性平台。这类系统普遍以“代码提交—编译运行—测试用例比对—结果判定”为核心流程，能够稳定支撑大规模刷题需求，但其评测大多仅停留在“对错判定”层面，缺乏对代码质量、复杂度与可读性的深度分析，题目资源也以人工维护的固定题库为主。

近年来，大语言模型（LLM）技术快速突破，在代码理解、代码生成、缺陷诊断、自然语言解析等方面展现出强大能力，为编程教育的智能化、个性化提供了

全新可能。将传统 OJ 的确定性评测与 AI 大模型的智能分析相结合，成为编程学习平台演进的重要方向。

在工程实现层面，前后端分离架构、容器化沙箱隔离、无状态令牌认证等已成为主流技术选型。本项目后端采用 Spring Boot 2.7.18 + MyBatis + MySQL 8.0 技术栈，使用 JWT 实现无状态认证；前端采用 Vue 3 + TypeScript + Vite + Pinia 构建单页应用，并集成 Monaco Editor 在线代码编辑器；评测引擎基于 Docker 容器实现沙箱隔离，支持 C++、Java、Python 多语言；AI 能力通过对接火山引擎方舟（Ark）大模型平台实现。上述技术现状为本平台“双引擎”架构的落地提供了坚实基础。

2.1.3 项目的主要开发工作

围绕“传统 OJ 判题 + AI 智能赋能”的双引擎定位，本项目的主要开发工作包括以下方面：

- 标准化算法题库与在线练习：构建按知识点、难度分级的结构化题库，提供在线代码编辑、题目收藏、历史练习、错题归集等功能。
- 自动化代码评测引擎：集成自研评测程序，实现代码自动编译、沙箱安全运行、测试用例逐组比对，精准返回 AC/WA/CE/RE/TLE 等专业评测结果。
- AI 多维度代码智能评价：接入大模型接口，对提交代码进行时间复杂度、空间复杂度、代码风格、可读性分析与优化建议、缺陷诊断。
- AI 智能生成算法题目与样例：根据用户指定的知识点、难度自动生成原创题目，并配套生成题目描述、输入输出格式、测试用例与参考题解。
- 学习数据可视化管理：留存提交记录、评测结果与 AI 评价报告，统计刷题量、正确率、薄弱知识点等并可视化展示。
- 安全沙箱与系统支撑：基于 Docker 容器实现代码隔离运行，配套用户认证、权限控制、文件存储（阿里云 OSS）、管理员后台等支撑能力。

2.1.4 项目组成员、分工及完成情况

本项目团队共 7 人，采用扁平化协作模式，成员专业互补、责任到人，覆盖产品需求、前后端开发、评测引擎、AI 对接、测试与文档全流程。各成员分工及完成情况如下表所示。

成员	主要角色	主要职责	完成情况
李俊熠	项目组长	统筹项目全局，把控进度、质量与预算；主导技术选型与架构设计；对接答辩评审与风险管控	已完成
冯朝航	需求 / 后端	开展需求调研、撰写需求规格说明书与 SDD 首版；部署沙箱安全环境、对接 AI 大模型接口	已完成
胡阳	后端 / 前端	搭建 Spring Boot 后端框架，开发业务接口，设计 MySQL 数据库，实现题库管理、用户管理等模块	已完成
唐代鑫	后端 / 评测	集成 C++ OJ 评测引擎，对接 AI 大模型接口，实现代码评测核心逻辑	已完成
黄嘉鑫	前端	开发在线代码编辑器，搭建前端页面与交互逻辑，实现学习数据可视化模块	已完成
谢浩敏	后端	参与后端业务接口开发与模块联调，协助评测与数据相关功能实现	已完成
孙赫	测试 / 文档	主笔需求分析文档、修订 SDD，编写测试设计文档、测试文档、用户操作手册与测试报告；设计并执行功能、接口、性能、安全测试，跟踪缺陷闭环	已完成

2.2 项目策划、分析与设计工作情况

2.2.1 项目立项计划及完成情况

项目在立项阶段完成了需求确认、团队分工、技术选型、开发环境搭建与预算编制，并形成《立项计划书》。项目定位为基于 Spring Boot 框架开发的 Web 端轻量化算法学习平台，采用“传统 OJ 判题 + AI 智能赋能”双引擎模式；总体目标是搭建安全稳定的在线评测环境，实现“AI 出题 → 在线做题 → 自动判题 → AI 评题 → 优化改进”的全流程闭环。

项目明确了范围边界：核心包含标准化题库、在线代码编辑器、自研 C++ 评测引擎、AI 大模型接口对接、学习数据可视化与安全沙箱环境；暂不包含移动端 APP、大规模商业化运维、付费会员体系等。项目预算遵循低成本、专款专用原

则，预估总预算约 ¥800–970 元，主要用于域名、云服务器、AI API 调用等，与 12 周开发周期同步管控。

项目总开发周期为 12 周，划分为四大里程碑节点，各阶段任务、交付物与完成情况如下表。

阶段	周期	核心任务	主要交付物	完成情况
立项筹备期	第 1 周	需求确认、团队分工、技术选型、环境搭建、预算编制	立项计划书、需求规格说明书、产品原型、技术方案	已完成
设计开发期	第 2–8 周	数据库设计、前后端开发、OJ 引擎集成、AI 接口对接、模块联调	可运行测试版本、完整源码、开发文档	已完成
测试优化期	第 9–11 周	全量功能测试、性能优化、缺陷修复、内部验收	测试报告、验收报告、优化后版本	已完成
结题交付期	第 12 周	系统部署上线、文档终稿、答辩材料准备	完整源码、用户手册、部署手册、结题报告、演示视频	已完成

总体而言，各里程碑节点均按计划达成，立项阶段设定的功能目标、交付物与开发周期得到了较好落实，为后续答辩与结题奠定了基础。

2.2.2 系统需求分析工作和结果

需求分析阶段输出了《需求分析文档》，清晰、完整地描述了平台的建设目标、功能要求、使用规则与验收标准，作为开发、测试与验收的统一依据。文档首先界定了平台的核心业务流程，包括普通用户刷题流程、管理员后台管理流程与代码判分流程三条主线。

在用户角色方面，平台划分为普通用户与普通管理员两类。普通用户面向在校学生、编程算法爱好者与求职备考人群，核心诉求是便捷查找题目、在线编写代码、快速获取准确判分结果，并借助 AI 获得代码优化、错题解析与编程答疑；管理员负责题库内容与日常运营管理，包括题库全流程管理、分类与测试用例维护、AI 内容审核、用户账号管理与运营数据查看。

在功能需求方面，文档梳理出 20 项用户需求（UR-001 ~ UR-020）并细化为覆盖六大模块的功能需求，主要模块与功能如下表所示。

功能模块	主要功能
用户管理	账号注册、登录、忘记密码、个人信息管理、密码修改、提交记录管理、题目收藏、错题本、账号注销
题库管理	题目分类管理、题目基础信息管理、测试用例管理、官方题解管理、题目前端展示与检索
在线编程与自动判分	在线代码编辑器、代码提交与预处理、自动判分核心、判分结果展示
AI 智能辅助	AI 代码多维评价、AI 智能题解、AI 个性化题目生成、AI 编程答疑、内容合规校验
数据统计	个人学习数据统计、平台运营数据统计与可视化
后台管理	用户与权限管理、AI 内容审核、平台配置与运行状态监控

在非功能需求方面，文档对性能、安全与易用性提出了量化指标：普通查询操作响应时间不超过 300ms，写入操作不超过 1s；常规题目（≤10 个测试用例）从提交到返回判分结果不超过 8s、平均不超过 3s，判分准确率 100%；AI 功能平均响应不超过 10–15s，调用成功率不低于 99%；系统支持约 1000 用户在线、200 人并发。安全方面要求密码加密存储、越权操作 100% 拦截、代码运行沙箱隔离、AI 与用户内容合规审核。文档最后建立了用户需求与功能需求的对应关系表及功能、性能、安全、交付四类验收标准，形成了可落地、可验证的需求基准。

2.2.3 系统设计工作及结果

系统设计阶段输出了《软件设计说明书（SDD）》，作为需求文档的配套技术基准，完整定义了平台的架构设计、模块划分、核心逻辑、接口规范、数据结构与部署方案。

在总体架构上，系统采用前后端分离的五层单向依赖架构，自上而下依次为表现层（Vue 刷题端 / 管理端 / Monaco 代码编辑器）、接入层（Nginx 反向代理、限流、跨域、HTTPS）、应用服务层（Spring Boot 核心业务模块、权限与事务）、基础设施层（OJ 引擎、Docker 沙箱、AI / 邮件客户端、缓存 / DAO）与数据存储层（MySQL、文件存储、Redis）。为凸显项目“判题引擎”与“AI 大模型”双核心，进一步将核心引擎层（C++ 自研 OJ 调度器、大模型 Prompt 编排与解析引擎）从通用设施层中拆分解耦。

系统按职责划分为六大独立功能模块：用户管理模块、题库管理模块、在线编程与自动判分模块、AI 智能辅助模块、数据统计模块与后台管理模块，模块间通

过标准化接口交互，边界清晰、职责单一。设计中还明确了若干关键设计决策，如采用 Docker 容器化沙箱实现代码运行隔离、采用异步任务队列 + WebSocket 实时推送判分流程、AI 模块与核心业务完全解耦、数据统计采用离线定时任务方案等。

在数据设计方面，遵循数据库三大范式，针对高频字段建立索引，敏感字段加密存储，统一下划线命名规范。系统核心数据表围绕用户、题目、提交、评测、统计等实体设计，主要数据表如下表所示。

数据表	核心用途
user	用户基础信息（用户名、密码、邮箱、角色等）
user_token	JWT 令牌存储与过期管理
category / problem / problem_category	题目分类、题目信息及题目—分类多对多关联
problem_test_case	题目测试用例（输入/输出文件 URL）
submission	代码提交记录（代码、语言、评测状态、耗时、内存）
evaluation	AI 代码评价报告（时间/空间复杂度、风格、可读性、优化建议）
solution	题目官方题解与参考代码
user_problem / user_statistics	用户题目通过状态与刷题统计

在接口设计方面，常规业务接口采用 HTTPS RESTful 协议，实时推送采用 WebSocket，统一响应格式为 {code, msg, data}，并规范了请求方法、状态码与权限校验规则。设计中还定义了代码提交状态机（待提交 → 待判分 → 判分中 → 终态）与题目状态机（草稿 → 待审核 → 已上线 → 已下架 → 已删除），明确了状态流转规则与约束。整体设计 100% 覆盖了需求文档的功能与非功能要求。

2.2.4 系统具体实现工作及结果

系统在设计方案的指导下完成了前后端及评测引擎的具体实现，技术栈与设计选型保持一致。

后端基于 Spring Boot 2.7.18 实现，采用 Controller—Service—Mapper 分层结构，使用 MyBatis 与 PageHelper 操作 MySQL 数据库，通过 JWT 与 TokenInterceptor 实现无状态认证与权限拦截，并由 GlobalExceptionHandler 统一处理异常。后端按模块实现了认证（AuthController）、用户（UserController）、题库（ProblemController）、评测（JudgeController / JudgeV1Controller）、AI 与算法

(AlgorithmController)、管理员 (AdminController)、系统资源监控 (SystemResourceController) 等控制器，共约 100 个 Java 源文件，覆盖六大业务模块约 60 余个 RESTful 接口。

前端基于 Vue 3 + TypeScript + Vite 构建单页应用，使用 Pinia 进行状态管理、Vue Router 管理路由、Axios 进行接口调用，并集成 Monaco Editor 实现在线代码编辑。前端实现了登录、注册、题库列表、题目详情、提交记录、提交详情、个人中心、AI 实验室等用户端页面，以及题目管理、分类管理、用户管理等管理员后台页面。

评测引擎方面，基于 Docker 容器构建安全沙箱，分别为 C++、Java、Python 构建独立判题镜像，配套 judge_cpp.sh、judge_java.sh、judge_python.sh 等判题脚本；每次评测在独立容器中编译运行用户代码，限制 CPU、内存与运行时间，逐组比对测试用例并返回结果，运行结束后立即销毁容器。AI 能力通过对接火山引擎方舟 (Ark) 大模型实现代码评价与题目生成，头像等文件通过阿里云 OSS 存储。系统最终实现了从用户注册、题目浏览、在线编码、代码提交、自动判分、AI 评价到学习数据统计的完整业务闭环。

2.3 系统部署与测试工作情况

2.3.1 系统部署与调试

系统采用单服务器集中式部署，所有组件运行于同一台 Linux 云服务器，由 Nginx 承载前端静态资源并作为后端反向代理，Spring Boot 以 Jar 包 + 系统服务方式运行于 8080 端口，MySQL 存储结构化业务数据，Docker Daemon 承载评测沙箱环境，后端通过 Docker API 动态创建与销毁评测容器。推荐服务器配置为 4 核 CPU、8GB 内存、200GB SSD。

部署与调试的主要流程包括：创建 oj_platform 数据库并导入表结构与初始化数据；修改 application.yml 中的数据库连接、判题脚本路径等配置，并配置阿里云 OSS 与火山引擎 Ark API Key 等环境变量；在 Docker 判题目录下分别构建 C++、Java、Python 三种判题镜像并验证；通过 Maven 打包并启动后端服务，访问

Knife4j 接口文档页面验证后端可用；安装前端依赖、配置后端代理并启动前端开发/构建服务。部署完成后对线上环境进行冒烟测试，确认 Nginx 反向代理、数据库与 Docker 判题服务连通正常。调试过程中重点解决了本地与云端 Docker 性能差异导致的冷启动判分耗时差异等问题，整体业务逻辑闭环未受影响。

2.3.2 系统测试

测试阶段输出了《测试设计文档》与《测试文档》，前者定义测试计划、测试需求分解与通过准则，后者记录全部测试用例及其执行结果。测试采用 IADT 方法（Input 输入—Action 处理—Data 数据—Test 输出）对各模块测试需求进行分解，确保测试点可追溯。

测试按阶段推进，依次开展单元测试、接口测试、集成测试、系统测试、性能测试、安全测试与回归测试，并使用 Postman/Apifox、JMeter、JUnit、Navicat 等工具配合执行。测试范围覆盖认证管理、用户个人中心、题库管理、在线编程与评测、AI 模块、管理员后台六大功能模块，以及数据库、前端界面、性能与安全等专项。功能测试对每个用例执行正常、异常、边界三类场景，主要测试结果如下表所示。

测试类别	覆盖内容	结果
功能测试	认证、用户、题库、评测、AI、管理员六大模块，约 70+ 功能用例	全部通过
接口测试	六大模块 60+ RESTful 接口的参数、响应、状态码与业务逻辑	通过率 ≥98%
数据库测试	唯一性约束、外键约束、统计一致性、密码加密等	全部通过
前端测试	页面渲染、交互、响应式设计、浏览器兼容性	通过率 ≥95%
性能测试	登录、题目列表、代码提交、AI 生成的并发与响应时间	指标达标
安全测试	JWT 校验、权限控制、SQL 注入、XSS、文件上传、沙箱隔离	全部通过
回归测试	核心业务流程在缺陷修复后的全面回归	全部通过

性能测试结果显示：100 并发用户登录平均响应约 650ms，题目列表平均响应约 480ms，50 并发代码提交平均响应约 1.8s，20 并发 AI 生成题目平均响应约 7.5s；500 用户并发场景下系统稳定运行，CPU 使用率约 75%、内存约 68%，连续运行 72 小时无内存泄漏。安全测试验证了 JWT 伪造/过期 Token 被拒绝、普通用户越权被拦截、SQL 注入与 XSS 被有效防护、Docker 沙箱对 rm -rf、fork 炸弹等

恶意代码 100% 拦截。测试结束时全部计划用例执行完毕、通过率满足准入准则，缺陷解决率超过 95%，无遗留严重缺陷，系统达到准出条件。

2.4 项目管理相关工具使用情况

为保障团队协作高效、流程规范，项目结合技术栈统一了开发与管理工具，覆盖代码管理、任务协作、接口测试、性能测试、设计建模与 AI 对接等环节，主要工具及用途如下表所示。

工具类型	选型工具	核心用途
版本控制	GitHub	代码托管、分支管理、协同开发与版本回溯
项目协作	飞书、语雀	任务分配、进度同步、文档共享与团队沟通
开发 IDE	IntelliJ IDEA、VS Code、CLion	后端、前端、C++ 评测引擎代码编写与调试
接口与测试	Postman、Apifox	接口调试、自动化接口测试与缺陷跟踪
性能测试	JMeter	并发与性能压力测试
设计建模	draw.io、Figma/飞书	架构图、流程图、产品原型与页面交互设计
数据库工具	Navicat / DBeaver	数据库可视化管理、表结构设计与数据调试
AI 对接	火山引擎方舟（Ark）、Apipost	大模型接口调试、代码分析与题目生成对接

在项目管理过程中，团队通过 GitHub 进行代码托管与分支协作，通过飞书/语雀同步任务与进度并沉淀文档，采用每日短会同步进展、每周例会全员对齐的节奏，并通过周报记录个人工作、现存问题与改进建议，形成了可追溯的过程管理资料。

2.5 项目讨论与体会

2.5.1 对项目过程的体会

通过本项目的完整实践，团队对软件项目从立项、需求、设计、开发、测试到部署交付的全生命周期有了系统而深入的认识。前期充分的需求分析与架构设计为后续开发奠定了清晰的基础，使各模块边界明确、协作顺畅；“双引擎”架构中将 AI 模块与核心业务解耦的设计，有效保证了在 AI 服务波动时核心判题流程仍可正常运行，体现了良好的可靠性设计思想。

在开发协作方面，团队体会到文档先行与接口规范统一的重要性。需求文档、SDD 与接口文档作为统一基准，显著降低了前后端联调时的沟通成本；而“测试左移”的实践——在开发早期即介入测试用例设计、推动接口异常示例补全——帮助团队更早发现并规避了状态码不一致、异步队列任务丢失、沙箱资源未释放等问题。此外，使用 Mock 数据解耦前端与后端进度、通过周报持续暴露风险等做法，也提升了整体协作效率。

2.5.2 项目管理过程优点与不足

优点方面：其一，项目管理流程较为完整，立项即明确了里程碑、分工、预算与风险应对方案，并按四阶段稳步推进，节点按期达成；其二，过程资料规范齐全，需求、设计、接口、数据库、测试等文档与源码版本保持一致，并通过周报形成了持续的过程记录；其三，工具链统一高效，GitHub、飞书、Postman、JMeter 等工具的配合保障了协作与质量；其四，质量管控到位，测试覆盖功能、性能、安全多维度，缺陷实现闭环管理，核心功能在交付前完成全量回归。

不足方面：其一，前期部分模块（如 Docker 沙箱环境）搭建进度滞后，一度阻塞核心判分流程的联调与预测试，反映出对核心技术难点的前置验证（POC）投入仍可加强；其二，部分异常场景（如恶意代码攻击）的测试点在早期未能完全穷举，安全渗透测试相对靠后；其三，第三方 AI 接口在高峰期偶发超时熔断、免费额度有限，对演示稳定性带来一定风险，提示在依赖外部服务时应更早设计降级与兜底机制；其四，前后端开发节奏存在阶段性不同步，导致部分测试只能脱离页面进行接口“盲测”。上述不足为后续项目在风险前置、资源预估与外部依赖管理方面提供了改进方向。

2.6 参考资料

以下参考资料以配置库（代码仓库 Docs 目录）上文件为准，正式文件随报告附后。

序号	文档名称	说明
(1)	项目策划 / 立项计划书	《01-立项计划书》：项目背景、定位、范

		围、分工、预算、里程碑与风险
(2)	系统需求规格说明书 (SRS)	《02-需求分析文档》：用户需求、功能需求、性能与安全需求、验收标准
(3)	系统概要设计文档 (SDD)	《06-系统设计文档 (SDD)》：架构、模块、数据、接口、状态机与部署设计
(4)	详细设计说明书 (可选，分模块)	《03-接口文档》《05-核心数据实体类文档》：接口规范与核心实体类设计
(5)	系统测试文档 (测试计划和测试程序/用例)	《07-测试设计文档》《08-测试文档》：测试计划、IADT 分解与全部测试用例及结果
(6)	用户文档说明书	《README》及《用户操作手册》：软件技术描述、安装部署与使用说明
(7)	项目管理过程文档 (可选)	Docs/Weekly_Work_Report 各成员个人周报、问题报告等过程管理资料
	数据库文档	《04-数据库文档》：数据库建库与建表 SQL 脚本